

Process Scheduling

Ch7

Sk Subidh Ali
IIT Bhilai

CPU Scheduling

Why scheduling: You have more number of processes to run than number of CPUs.

What is CPU Scheduling: It is the policy for deciding, at any given moment, **which process to assign to CPU and for how long.**

Purpose of Scheduler:

1. **Which task runs next?** (selection policy)
2. **For how long before it's reconsidered?** (time-slicing / preemption policy)
3. **When does the decision get revisited?** (triggering events)

Best scheduling Algo: It is still a theoretical domain of study

1. **Simple *FCFS, SJF:** theoretically provable, easy to implement but less efficient
2. **Complex:** may be very efficient no theoretical guarantee

When to invoke Scheduler

Modern scheduler are invoked based on interrupt or trap

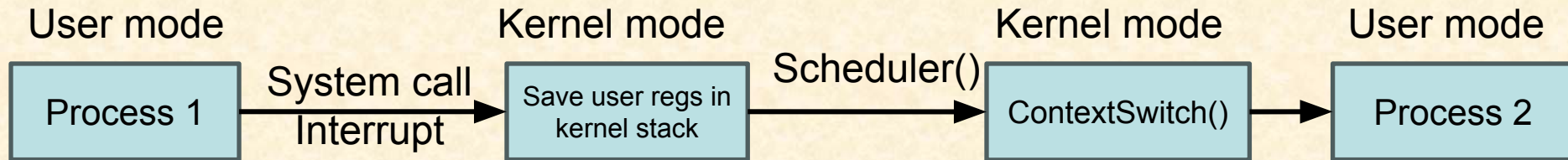
In XV6, scheduler runs in infinite loop

- **Timer interrupt:** A process completes its allocated CPU time. Timer interrupt invokes the scheduler
- **A process goes to sleep/blocked/waiting state:** scheduler invoked based on corresponding sleep, blocked, waiting system call
- **A process goes to Sleep/Block/waiting to ready state:** the device for which process was waiting will interrupt on completion, which triggers the scheduler
- **A process completes (exit()) or terminated (kill()):** trap call will trigger the scheduler
- **A higher-priority process arrives:**

What happened when Scheduler is invoked

Scheduler:

1. Choose the next ready process from the list of process
2. Context switch:
 - Mark the state of current process accordingly
 - Save it's context in its PCB
 - Mark the state of the next process as running
 - Load the context of next process to CPU and resume its execution



Preemptive vs. non preemptive

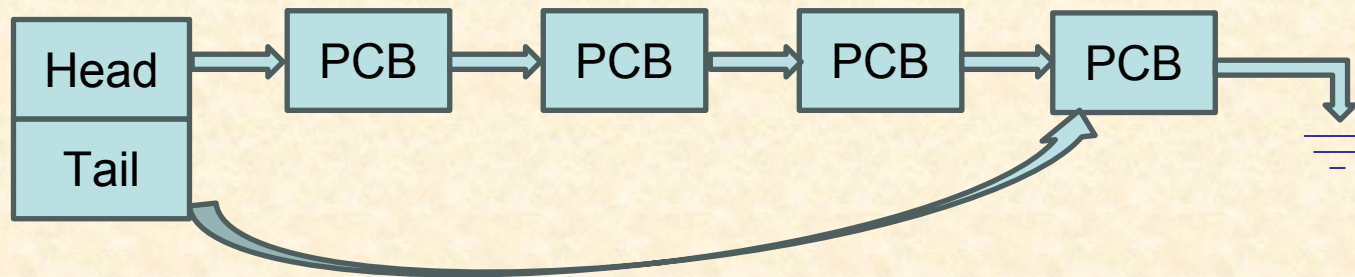
- **Non preemptive:** process continue executing in CPU until it voluntarily gives it up (exit() or kill()).
 - Process may infinite-loop and hold CPU foreve
- **Preemptive scheduling:** the OS can forcibly interrupt a running process and hand the CPU over to other process.
- Modern scheduler uses preemptive scheduling

Goal of CPU scheduling policy

- **Maximize**
 - **CPU utilization:** CPU is precious. Should not be sitting idle
 - **Throughput:** total work done per unit time
- **Minimize:**
 - **latency/responsiveness:** how quickly a given process start executing. Amount of time it has to wait before assigned to CPU
 - **Turnaround time:** time from creation to completion
- **Fairness:** everyone gets a fair share of CPU
- **Simplicity/predictability, low overhead, fast responsive**

Data structure for scheduling

Ready state: simple solution is queue



Linux: uses Red-black-Tree for its *Completely Fair Scheduling*

Windows: uses Multi-level priority queue, with 32 queues

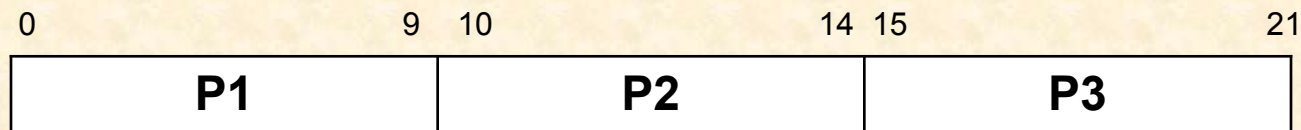
For waiting queue almost every OS uses regular queue

FIFO/FCFS: First come first serve

- New or ready processes are kept in a ready queue
- **Scheduler** executes the first process from queue
- Non-preemptive: every process allowed to run till it completes/terminates/block/waits.

Example: At any specific time three processes are in ready queue:
P1=>P2=>P3

Process	CPU time needed (unit)	Execution time
P1	10	0-9
P2	5	10-14
P3	7	15-21

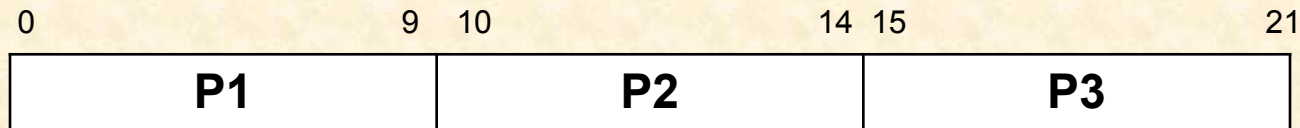


Gantt chart

Lets See the performance FIFO

CPU utilization: 100% as CPU was never idle

Process	CPU time/burst time needed (unit)	Execution time (schedule)
P1	10	0-9
P2	5	10-14
P3	7	15-21



Schedule

Lets See the performance FIFO

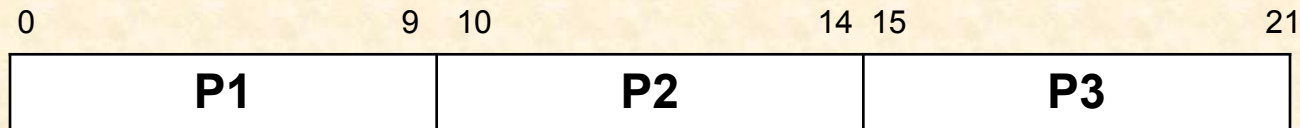
Average Latency/waiting time: $\frac{0+10+15}{3} = 8.3$

P1: 0 waiting

P2: 10 unit waiting

P3: 15 unit waiting

Process	CPU time/burst time needed (unit)	Execution time (schedule)
P1	10	0-9
P2	5	10-14
P3	7	15-21



Schedule

Lets See the performance FIFO

Turn around time:

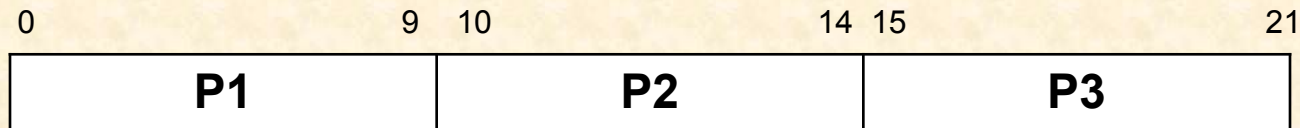
$$\frac{10+15+22}{3} = 15.6$$

P1: created at 0 ends at 14

P2: created at 0 ends at 15

P3: created at 0 ends at 21

Process	CPU time/burst time needed (unit)	Execution time (schedule)
P1	10	0-9
P2	5	10-14
P3	7	15-21



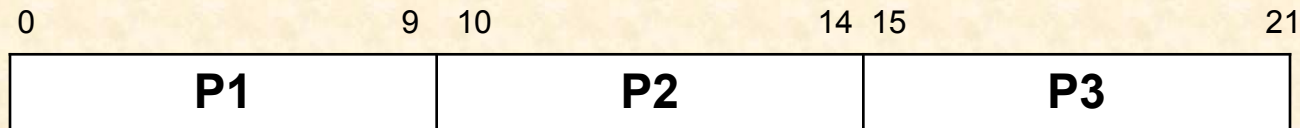
Schedule

Score card of FIFO

Average waiting time: 8.3

Turn around time: 15.6

Process	CPU time/burst time needed (unit)	Execution time (schedule)
P1	10	0-9
P2	5	10-14
P3	7	15-21



Schedule

What happens when P1 is large?

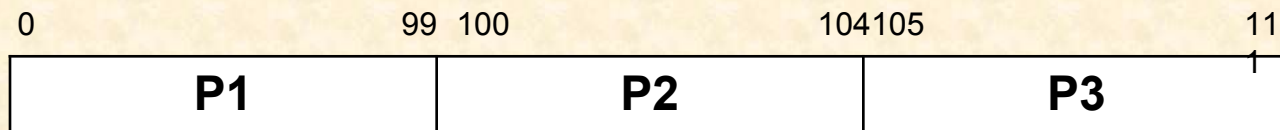
Can you find the following???



Average waiting time: ?

Turn around time: ?

Process	CPU time/burst time needed (unit)	Execution time (schedule)
P1	100	0-99
P2	5	100-104
P3	7	105-111



Schedule

What happens when P1 is large?

Average waiting time: $\frac{0+100+105}{3} = 68.3$

Turn around time: $\frac{100+105+112}{3} = 105.66$

Convoy effect: large process can cause excessive delay to small process

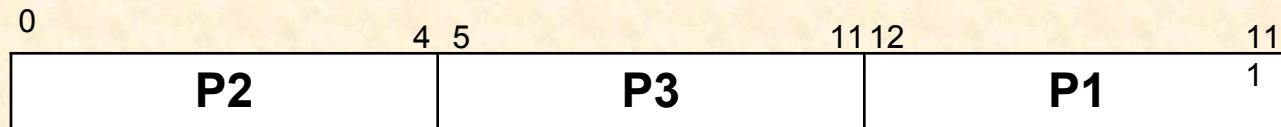


SJF: Shortest Job First

- **Scheduler** executes process with smallest *burst time*
- Non-preemptive:

Example: At any specific time three processes are in ready queue:
P1=>P2=>P3

Process	CPU time/burst time needed (unit)	Execution time (schedule)
P1	100	12-111
P2	5	0-4
P3	7	5-11



Gantt chart

SJF: performance

Can you find the following???



Average waiting time: $\frac{0+5+12}{3} = 5.6$

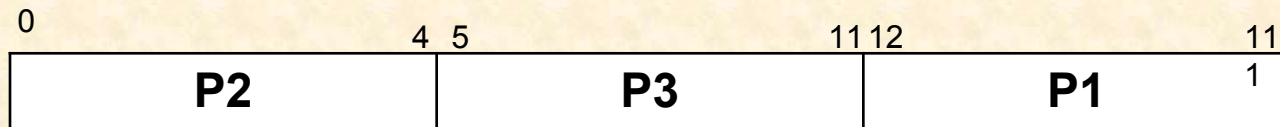
Turn around time: $\frac{5+12+112}{3} = 43$

What if P2 arrives just after P1 start executing?



Convoy effect still exist in some cases

Way better than FIFO. No convoy effect

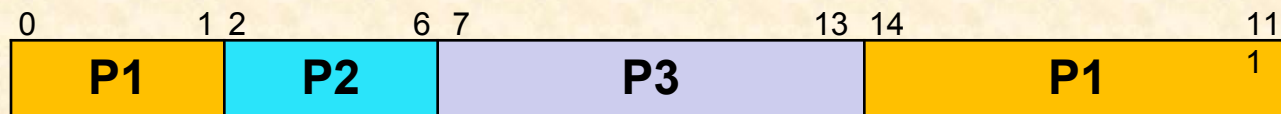


Gantt chart

SRTF: Shortest Remaining Time First

- SJF with preemptive scheduling
- New process with small remaining burst time can pre-empt running process with bigger burst time
- Get rid off convoy effect

Process	Arrival time	CPU burst time
P1	0	100
P2	1	5
P3	2	7



Gantt chart

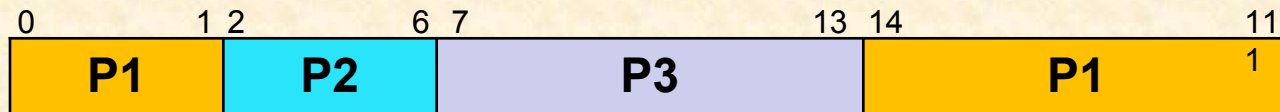
SRTF: Shortest Remaining Time First

Average waiting time: $\frac{0+5+12}{3} = 5.6$

Turn around time: $\frac{11+2+5+11}{3} = 42.6$

Process	Arrival time	CPU burst time
P1	0	100
P2	1	5
P3	2	7

P1 preempted as P2 requires smaller time (5 unit) to complete



Gantt chart

Next class: tutorial

Next theory class: Advanced scheduling: Ch 8